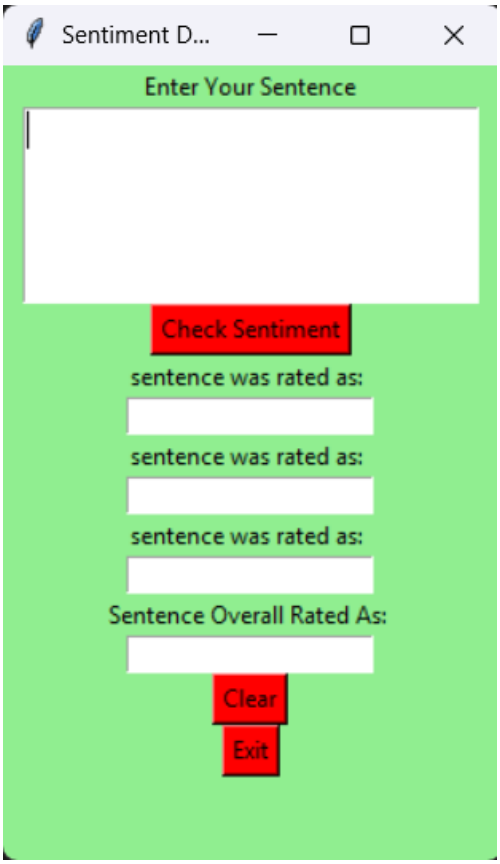
Sentiment Detector GUI



Overview

This Python application is a Sentiment Detector GUI built using Tkinter and the VADER Sentiment Analysis library. The application allows users to input a sentence and detect the sentiment of the text (positive, negative, or neutral) along with individual sentiment scores for each category (negative, neutral, and positive). The application displays the sentiment results in a graphical user interface (GUI).

Libraries Used

- vaderSentiment: This is a Python library that provides a sentiment analysis tool. The SentimentIntensityAnalyzer class from the library is used to analyze the sentiment of a given text.
- tkinter: This is the standard GUI toolkit for Python, used for building the graphical interface of the application.

```
pip install vaderSentiment
```

1. Importing Libraries

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from tkinter import *
```

- **VADER Sentiment Analysis Library**: The vaderSentiment library is imported to analyze the sentiment of the input sentence. The SentimentIntensityAnalyzer class provides the method polarity_scores() which returns sentiment scores (positive, negative, neutral, and compound).

- **Tkinter Library**: Tkinter is a built-in Python library used to create graphical user interfaces (GUI). The * means all classes and methods from Tkinter are being imported for use in the code.

2. clearAll Function

```
# Function for clearing the contents of all entry boxes and text area.
def clearAll() :

    # deleting the content from the entry box
    negativeField.delete(0, END)
    neutralField.delete(0, END)
    positiveField.delete(0, END)
    overallField.delete(0, END)

    # whole content of text area is deleted
    textArea.delete(1.0, END)
```

Purpose: This function is used to clear the contents of all the entry fields and text area in the GUI. When invoked, it deletes the text from the following:

- negativeField, neutralField, positiveField, and overallField (which display sentiment results).
- textArea (the area where the user enters the sentence to analyze).

3. detect_sentiment Function

```
# function to print sentiments of the sentence.
def detect_sentiment():

    # get input
    sentence = textArea.get("1.0", "end")

    # SentimentIntensityAnalyzer object
    sid_obj = SentimentIntensityAnalyzer()

    # polarity_scores method of SentimentIntensityAnalyzer object gives a sentiment dictionary. Which contains pos, neg, neu, and compound scores.
    sentiment_dict = sid_obj.polarity_scores(sentence)

    string = str(sentiment_dict['neg']*100) + "% Negative"
    negativeField.insert(10, string)


    string = str(sentiment_dict['neu']*100) + "% Neutral"
    neutralField.insert(10, string)

    string = str(sentiment_dict['pos']*100) +"% Positive"
    positiveField.insert(10, string)

    # decide sentiment as positive, negative and neutral
    if sentiment_dict['compound'] >= 0.05 :
        string = "Positive"

    elif sentiment_dict['compound'] <= - 0.05 :
        string = "Negative"


    else :
        string = "Neutral"

    overallField.insert(10, string)
```

- **Purpose**: This function is called when the "Check Sentiment" button is clicked. It analyzes the sentiment of the text entered by the user:
    - It retrieves the text from textArea (the area where the user inputs the sentence).
    - It then uses the SentimentIntensityAnalyzer() object to analyze the sentiment of the sentence and returns a sentiment dictionary.
    - The sentiment dictionary contains four values: neg, neu, pos, and compound. The function processes each sentiment score and displays them in their respective entry fields (negativeField, neutralField, positiveField).
    - The compound score is a summary of the sentiment. Based on this score, the sentence is classified as "Positive", "Negative", or "Neutral", and the result is displayed in overallField.

4. GUI Setup and Configuration

```
if __name__ == "__main__" :


    # GUI
    gui = Tk()

    # background colour of GUI
    gui.config(background = "light green")

    # name of GUI window
    gui.title("Sentiment Detector")

    # configuration of GUI window
    gui.geometry("250x400")
```

- **Purpose**: This section initializes the Tkinter GUI window (gui). The following configurations are set:
    - Background color is set to "light green".
    - The title of the window is set to "Sentiment Detector".
    - The size of the window is configured to 250x400 pixels.

5. Adding Widgets (Labels, Text Area, Buttons)

```
# label
enterText = Label(gui, text = "Enter Your Sentence",
                  bg = "light green")

# a text area for the root
textArea = Text(gui, height = 5, width = 25, font = "lucida 13")

# create a Submit Button and place into the root window
check = Button(gui, text = "Check Sentiment", fg = "Black",
               bg = "Red", command = detect_sentiment)

negative = Label(gui, text = "sentence was rated as: ",
                 bg = "light green")

neutral = Label(gui, text = "sentence was rated as: ",
                bg = "light green")

positive = Label(gui, text = "sentence was rated as: ",
                 bg = "light green")

overall = Label(gui, text = "Sentence Overall Rated As: ",
                bg = "light green")
# text entry box
negativeField = Entry(gui)

neutralField = Entry(gui)

positiveField = Entry(gui)

overallField = Entry(gui)

clear = Button(gui, text = "Clear", fg = "Black",
               bg = "Red", command = clearAll)

Exit = Button(gui, text = "Exit", fg = "Black",
              bg = "Red", command = exit)
```

Purpose: This section creates the different GUI components:

- **Labels**: These are text elements that display descriptions like "Enter Your Sentence", "sentence was rated as", and "Sentence Overall Rated As".
- **Text Area**: textArea is where the user inputs a sentence to analyze.
- **Buttons**:
    - The Check Sentiment button triggers the detect_sentiment function to analyze the input sentence.

    - The Clear button clears all the fields and text areas using the clearAll function.
    - The Exit button closes the application.
- **Entry Fields**: negativeField, neutralField, positiveField, and overallField are the fields where the sentiment results are displayed.

6. Grid Layout for Organizing Widgets

```
# grid method for placing the widgets at respective positions in table like structure
enterText.grid(row = 0, column = 2)

textArea.grid(row = 1, column = 2, padx = 10, sticky = W)

check.grid(row = 2, column = 2)

negative.grid(row = 3, column = 2)

neutral.grid(row = 5, column = 2)

positive.grid(row = 7, column = 2)

overall.grid(row = 9, column = 2)

negativeField.grid(row = 4, column = 2)

neutralField.grid(row = 6, column = 2)

positiveField.grid(row = 8, column = 2)

overallField.grid(row = 10, column = 2)

clear.grid(row = 11, column = 2)

Exit.grid(row = 12, column = 2)
```

- **Purpose**: This section uses the grid method to organize the widgets in a tabular structure. Each widget (label, button, entry field) is placed in a specific row and column of the window. The padx argument adds horizontal padding, and sticky = W ensures the text in the textArea widget is aligned to the left.
-

7. Starting the GUI

```
# start the GUI
gui.mainloop()
```

Purpose: The mainloop() method is called to start the Tkinter event loop. It keeps the application running and allows the user to interact with the GUI. Without this, the window would appear and close immediately.

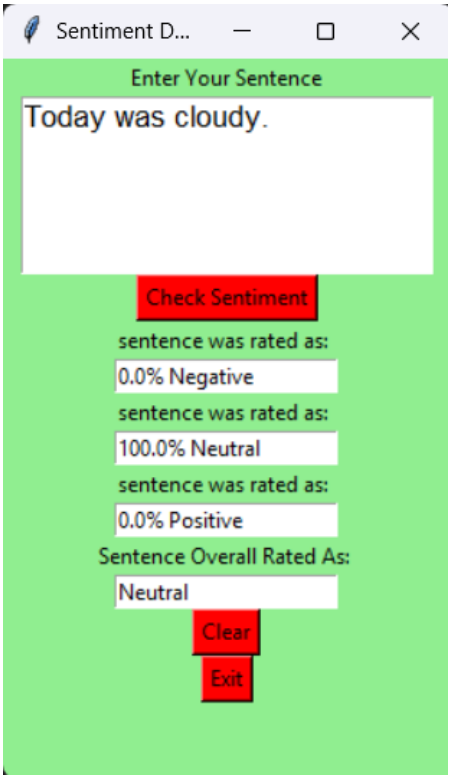
Summary:

- The code uses Tkinter to create a simple GUI for sentiment analysis.
- The vaderSentiment library is used to analyze the sentiment of a sentence provided by the user.
- The user can enter a sentence, click "Check Sentiment", and see the sentiment analysis results (positive, neutral, negative) displayed in entry fields.
- The "Clear" button resets all fields, and the "Exit" button closes the application.

This GUI is a basic sentiment analysis tool that can be extended with additional features or integrated into larger applications.

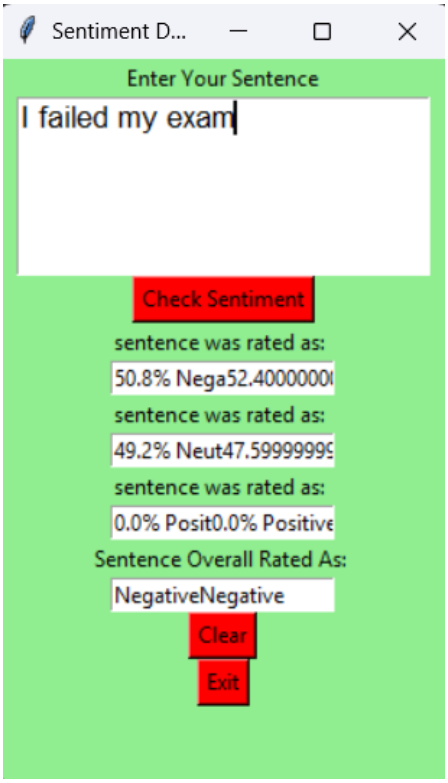
Final result:

- When you write a sentence with to sentiment such as ”Today was cloudy”:



This sentence has no sentiment so the code recognized it 100% neutral.

- I wrote a negative sentence in another example:



“I failed my exam” is between negative and neutral so the code recognized it 51% negative and 49% neutral.

- When I wrote “I was happy today” the program was able to recognize it was more positive so it returned 55% positive and 44% neutral.